Contents

Porting to Next.js with Vercel Cloud Deployment 2

Project Goal 2

Project Report Requirements 2

Part 1 3

Goal 3

Learn the git command line interface (CLI) 3

Next.js 4

Associating the GitHub Repository with a Next.js application. 4

Associating the GitHub Repository with a Vercel project. 5

Ongoing Workflow 6

Project Status 6

Part 2 Introducing PostgreSQL 7

Goal 7

Setting Up PostgreSQL Locally 7

Install PostgreSQL Locally 7

Connect Next.js App to Local PostgreSQL 8

Create and Populate the Local Database 9

Test Local Database Connection 11

Push to GitHub 11

Run the Deployed App (Expect Failure) 12

Project Status 12

Part 3 Music API in the Cloud 12

Goal 12

Provision Vercel Postgres (free tier) 12

Tie your local environment to your Vercel project 12

Managing your Environment Variables 13

Update Vercel Database to *music* 13

Database Connection Configuration for your Vercel project. 13

The Music Database on Vercel 14
Update *db-check* 15
Testing the Deployed Connection 15
Next.js API Creation 16
Testing the API Endpoints 16
Understanding the Next.js API 17
Architectural Critique 19
Project Status 20
Deliverables 20
Rubric 21

Porting to Next.js with Vercel Cloud Deployment

Project Goal

Port the localhost React application stack to the cloud using Next.js and Vercel.

- Our current stack is: Express + MySQL + React
- Our new stack is: Vercel + PostgreSQL + Next.js (React framework)
- Vercel is a cloud provider and the publisher of Next.js. Next.js is a framework designed with the cloud in mind.

This document covers the first part of the project, Music API deployment to the Vercel Cloud. The React UI port will be part 2.

Project Report Requirements

Throughout this activity, you will be asked to create screenshots showing the current state of the application.

1. *If the screenshot is a UI that you control, always include your roster name.*
2. *Section your screenshots following the part structure of this document.*
3. *Accumulate screenshots as you work in a Word document.*
    a. *Include the screenshots the order given.*
4. *Caption all screenshots*

Part 1

Goal

Introducing Next.js and Vercel Deployment

Next.js

Next.js is a powerful React framework designed to streamline the development of fast, scalable, and SEO-friendly web applications. It extends React by adding features like server-side rendering (SSR), static site generation (SSG), and incremental static regeneration (ISR)—all of which optimize performance and user experience. With its intuitive file-based routing system, built-in API routes, and automatic code splitting, Next.js allows developers to build full-stack applications with minimal configuration. Whether you're crafting dynamic dashboards or deploying static marketing pages, Next.js bridges the gap between frontend elegance and backend efficiency.

Vercel

Vercel is the creator of Next.js. As a cloud platform, Vercel is purpose-built to support Next.js and other frontend frameworks, offering instant deployments, preview environments, and global edge delivery with zero configuration. With built-in support for serverless functions, middleware, and seamless Git integration, Vercel turns every code push into a production-ready release.

Create a Vercel account as a Hobbyist (free tier). ***Use your GitHub credentials for an automatic tie to your source code repository.***

Prerequisites

GitHub is a specific installation of git. Make sure you install the CLIs (command line interface) for both Git (cleverly named 'git') and GitHub ('gh'). After installation open a terminal window and request versions:

```
PS C:\> git --version
git version 2.48.1.windows.1
PS C:\> gh --version
gh version 2.78.0 (2025-08-21)
https://github.com/cli/cli/releases/tag/v2.78.0
```

Figure 1 Installation Check

If you get a response from both CLIs, you're ready to proceed.

Learn the git command line interface (CLI)

The official tutorial is best:

https://git-scm.com/docs/gittutorial

GitHub

Create an empty repository on GitHub "cst391-music-app". You'll be using this name a lot so keep a copy at hand for pasting. I'm using that project name throughout this document.

Next.js

Create a default Next.js app:

```
npx create-next-app@latest cst391-music-app
```

When prompted, accept all the defaults

- Since we are using Bootstrap, decline Tailwind CSS.

Document all create-next-app prompts and summarize their purpose.

When you create a new Next.js app using create-next-app:

- A local Git repository has been initialized (git init)
- All project files are already committed as the first commit

---

After project creation modify a string in "page.tsx" to include your name. "npm run dev" in the project root folder. Follow the instructions to visit the page in the browser.

This simplifies the next step, pushing to a GitHub repository.

Associating the GitHub Repository with a Next.js application.

Now we are going to set the remote origin of the Next.js application to the empty remote repository created earlier.

git remote add origin https://github.com/<<your-username>>/<<your-repo-name.git>>

Replace <<your-username>> and <<your-repo-name>> with your actual GitHub info.

Run the resulting command in the Next.js project root.

Verify that this worked with:

git remote -v

Next push the current code to the GitHub server:

```
git push -u origin master
```

Examine the GitHub repository. You should now find it populated with the default Next.js app. The association has been made: As you modify the code locally, you'll be able to push your modifications to GitHub. These instructions follow.

Take a screenshot of your GitHub repository with default Next.js app. Make sure your account name is shown along with the code.

Associating the GitHub Repository with a Vercel project.

In your Vercel Projects page

1. Select Import Project.
2. Select 'Add GitHub account'
3. The next screen states 'Install Vercel'
    a. Choose "Only select repository"
    b. Select "<<your -account>>/cst391-music-app"
    c. "Install"
    d. "Import"
    e. "Deploy" (wait...)

Go to the Vercel project's Dashboard. On that dashboard find the link under the title 'Domains'. This is a public URL. Test the URL in a browser, you should see the default Next.js app.

Take a screenshot of the deployed default Next.js app. Make sure you show the address bar. Your name should be shown on the screen. This is already in place from the localhost step.

This completes the loop. When you create a new feature or fix a bug and push this code change to GitHub, your change will automatically be deployed by Vercel. It's a DevOps solution that has a lot of flexibility. Git is based on branching. Our simple workflow does not cover branching, but in a real project you should learn about branching and use it. Even for a single person project. If you use branching in git, you can configure Vercel to release private URLs for testing. This may not be on the free tier.

Vercel CLI

Later in this tutorial, you'll use the Vercel CLI. Since we're setting up Vercel in this step, install the Vercel CLI:

```
npm install -g Vercel
```

Login using the same email associated with your GitHub account and Vercel project:

```
vercel login
```

Use GitHub to authenticate. On successful completion, you'll be able to administer your Vercel project from the CLI.

Ongoing Workflow

Every change you make, you can now deploy. Deploy small changes:

```
Check status of changes
    git status
Stage all changes
    git add .
Commit with a message
    git commit -m "Your message here"
    It is very important that your commit message be both brief and descriptive. These
    commit messages will form a project change log for your team.
# Push to GitHub
    git push
```

Open 'page.tsx' and make a small change. Run the steps above to push this change to GitHub. With several minutes, Vercel will deploy the change. You can view the build status on both GitHub and Vercel. Test the public URL.

Take a screenshot of the deployed default Next.js app. The screenshot should show a difference from the earlier shot. Call out the difference.

Project Status

We have a deployed default Next.js application that will host both the Music API webserver and the React Music app.

Part 2 Introducing PostgreSQL

Goal

Establishing a local and cloud music database using PostgreSQL.

Connecting the default Next.js to the PostgreSQL music database locally and in the cloud.

This will set the table for Part 3, RESTful API development in the Next.js server context.

Setting Up PostgreSQL Locally

Overview

This guide walks you through:

- Installing PostgreSQL
- Running the musicDbPostgresSQL.sql script locally
- Connecting a Next.js app to the local database
- Pushing changes to GitHub
- Verifying failure in the deployed app
- Provisioning a PostgreSQL database via Vercel
- Running the same SQL script remotely
- Verifying success in the deployed app

Install PostgreSQL Locally

PostgreSQL is a powerful, open-source relational database. It's widely supported across platforms and ideal for full-stack development.

Why PostgreSQL? Vercel's free tier does not support MySQL. PostgreSQL is the default choice for relational data in Vercel-hosted apps.

1. Download PostgreSQL from the official site
2. Choose your operating system and follow the installer instructions
3. During setup:
    - Use default port 5432 (PostgreSQL's standard port)
    - Set username to postgres
    - Choose a password you'll remember

Connect Next.js App to Local PostgreSQL

1. In your Next.js project, install the PostgreSQL client:

   npm install pg

   npm install --save-dev @types/pg

2. Create a file at lib/db.js:

```
import { Pool } from 'pg';

// Persist across hot reloads in dev and across warm serverless invocations
let pool: Pool | undefined;

export function getPool(): Pool {
  if (!pool) {
    const URL = process.env.POSTGRES_URL ?? process.env.DATABASE_URL;
    if (!URL) throw new Error('POSTGRES_URL (or DATABASE_URL) not set');

    pool = new Pool({
      connectionString: URL,
      // TLS is required by many hosted PGs; harmless locally when undefined
      ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : undefined,
      max: 5, // small pool suits serverless
    });
  }
  return pool;
}

```

3. Add a ".env.local" file to the root of your project:

```
POSTGRES_URL=postgresql://postgres:yourpassword@localhost:5432/music
```

4. Add 'pages/api/db-check.ts'

```
// pages/api/db-check.ts
import type { NextApiRequest, NextApiResponse } from 'next';
import { getPool } from '@/lib/db';

const environment = process.env.NODE_ENV;
const dbUrl = process.env.DATABASE_URL;

export default async function handler(req: NextApiRequest, res:
NextApiResponse) {
```

```
  try {
    const db = getPool();
    const { rows } = await db.query('select now() as now');
    res.status(200).json({ time: rows[0], message: `Sparks Database
connection successful. Running in ${environment}. DATABASE_URL: ${dbUrl}` });
  } catch (err) {
    res.status(500).json({ error: 'Database connection failed',
       details: (err as Error).message, message: `Sparks Database connection
failed. Running in ${environment}. DATABASE_URL: ${dbUrl}` });
  }
}
```

5. Run the app locally and test the URL :

   npm run dev

   http://localhost:3000/api/db-check

6. Since we haven't set up our database yet, a connection failure is expected.

Test the 'db-check URL. Take a screenshot of your browser. This step should show failure to connect to the local PostgreSQL database.

Create and Populate the Local Database

1. Open pgAdmin 4
   - This is the workbench software installed with PostgreSQL.
2. When you supply the password you set when installed PostgreSQL you will connect to the DB server:

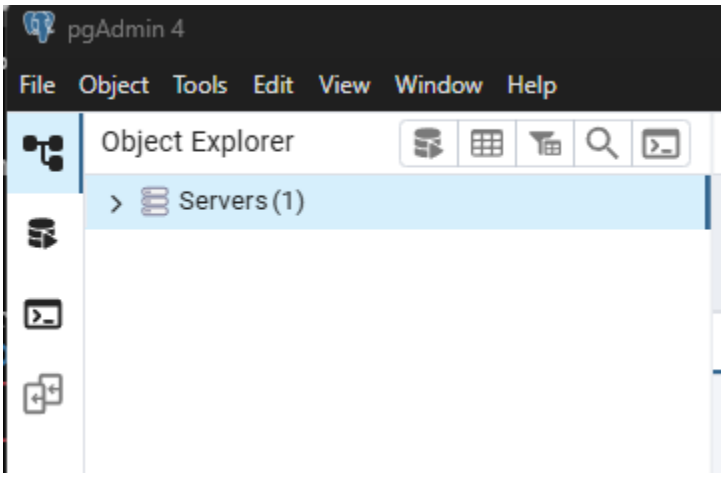

Figure 2 Connected Client-Server

3. Open the SQL Editor. Expand until you see "PostgreSQL 17" (the exact version may change) and Databases. Open the context menu of Database to see 'Create'. This will allow you to create the "music" database:

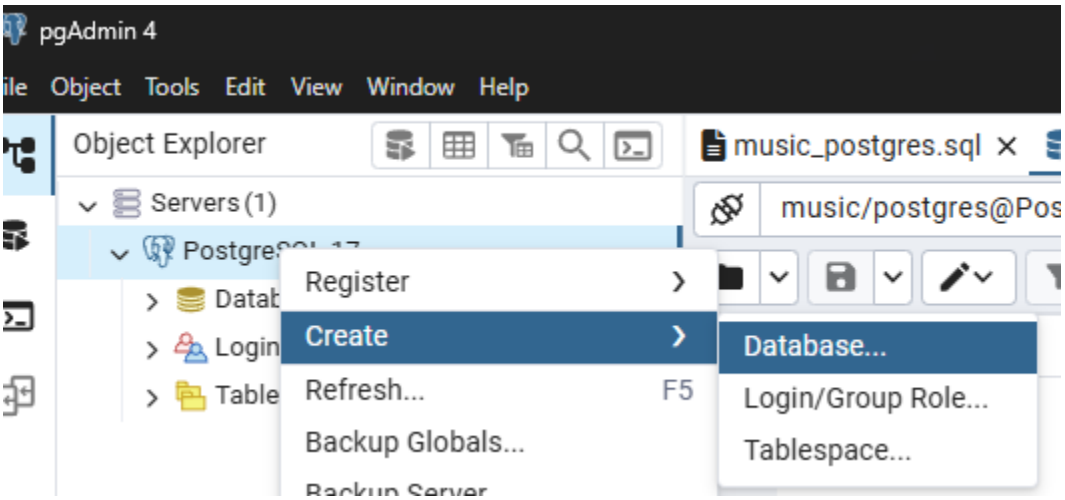

Figure 3 Database Creation

- Open the context menu on the music database. Pick Load and pick CREATE Script. This will open an editor with a starter script. Don't run the starter. Instead open the provided "musicDbPostgresSQL.sql" script. This will create a database named music and populate it with schema and data:

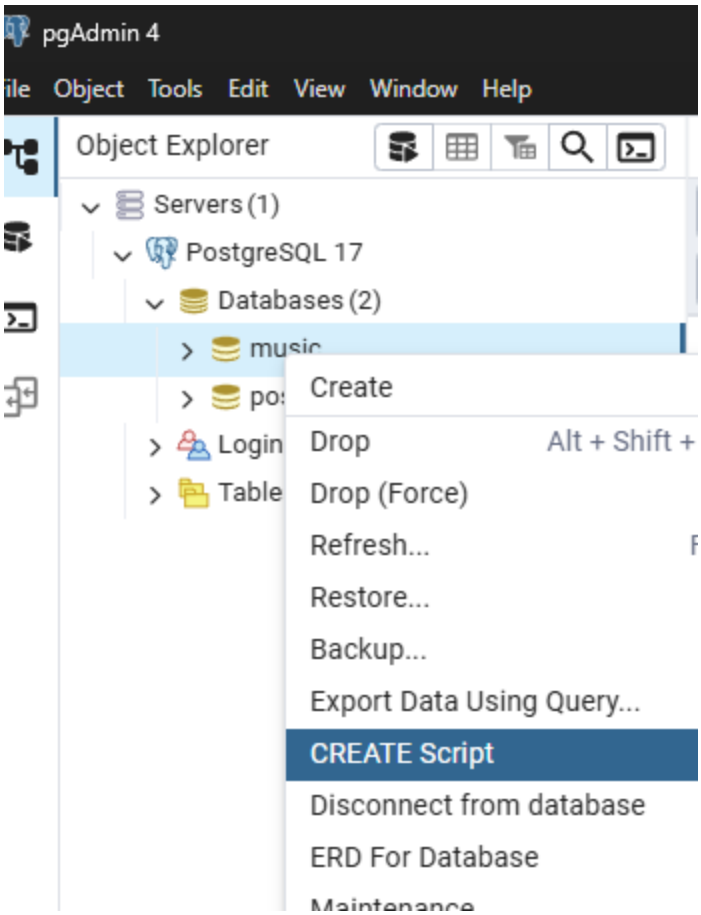

Figure 4 Create Script

Test Local Database Connection

You should now be able to connect to the *music* database. Rerun the URL tested earlier:

http://localhost:3000/api/db-check

Test the 'db-check URL. Take a screenshot of your browser. This step should show connection success to the local installation of Vercel. Make sure to modify the results so that your name is shown.

Push to GitHub

Once the local connection is working:

```
git add .

git commit -m "Connect to local PostgreSQL musicApp DB"

git push origin master
```

Run the Deployed App (Expect Failure)

Visit your deployed Vercel app. You should see a console error indicating failure to connect to the database.

Take a screenshot of your browser. Make sure the URL is part of the screenshot, it should show a Vercel address. This step should show failure to connect.

Project Status

We have a local Next.js application that connects to a local database. In the next step, we will connect to a Vercel cloud database.

Part 3 Music API in the Cloud

Goal

Recreate the Express music API in Next.js

Provision Vercel Postgres (free tier)

1. In your Vercel "cst-391-music-app" project, go to **Storage → Add** and choose **Neon**.
2. You need to connect the application to the newly provisioned database server.
    - In Storage, press '*Connect*'

Tie your local environment to your Vercel project

Earlier, you installed the Vercel CLI and authenticated it against your account with 'vercel login'. Now we will complete the database configuration:

1. **vercel link**
   Project Binding
   Links your local project folder to the corresponding Vercel project. This is the tether between your local code and its cloud incarnation.
2. **vercel env pull .env.development.local**
   Pulls environment variables from Vercel into a local *.env.development.local* file.

Run your app locally, it should report a successful connection to the PostgreSQL database *neondb* hosted on Vercel.

Take a screenshot of your application running locally and connecting successfully to the 'neondb' on Vercel.

Managing your Environment Variables

If you followed the earlier instructions, you should also have a *env.local* configuration file. Both are read automatically, but settings in *.env.development.local* take precedence. Take the following steps to unify your settings:

1. Copy into *.env.development.local* the POSTGRES_URL setting you created earlier to connect to localhost PostgreSQL server.
    a. Comment it out with a hash (#) character.
2. You can swap connections (localhost vs. Vercel deployment) by toggling the commented sections of. env.*development.local*.
    a. Later in this document, it will simply state,
        i. "toggle connection in *.env.development.local* to localhost", or
        ii. "toggle connection in *.env.development.local* to Vercel".

Once the *.env.development.local* config file is in place, you can delete *env.local*.

A Screenshot of /api/db-check reporting music database connection success on localhost

---

Update Vercel Database to *music*

Finally, update the *.env.development.local* configurations settings, replace *neondb with music*. Use whole word search and replace carefully.

Database Connection Configuration for your Vercel project.

Vercel injects environment variables into your application at runtime. Therefore, we don't have a physical environment file, but we still need to define those variables.

Vercel allows us to upload an env file. Create a ***.env*** file on your machine. The contents should come from the config information in *.env.development.local*. The variables you'll need to copy into in this file. **Make sure references to *neondb* are updated to *music*.**

DATABASE_URL
PGDATABASE
PGPASSWORD
PGHOST

Login to Vercel. Click on your **project** from the dashboard.

1. Click **Settings**.
2. Find the **Environment Variables** section.
3. Upload the .env file you created.

The Music Database on Vercel

Next, we will use *pgAdmin* to create the *music* database on the PostgreSQL deployment.

1. Run *pgAdmin*. Right click on the *Servers* and *Register* a server.
    a. Give the server a meaningful name *Vercel Database*.
    b. On the *Connection* tab use the settings from the *"Parameters for constructing your own connection string"* section of the *env.local* config file.
        i. *Host name/address* ← PGHOST_UNPOOLED
        ii. *Maintenance database* ← neondb
        iii. *Username* ← neondb_owner
        iv. *Password* ← PGPASSWORD
    c. Once the connection is working, create the 'music' database.
        i. Use *template0* when creating the database.
    d. Follow the instructions used to create the local version of *the database music on the Vercel deployment*. *See: <u>Create and Populate the Local Database</u>*

Take a screenshot of the music database on Vercel in pgAdmin

Update *db-check*

The updated *db-check.ts* code includes a simple query against the artist table. Change 'Sparks' to your roster name:

```
// pages/api/db-check.ts
import type { NextApiRequest, NextApiResponse } from 'next';
import { getPool } from '@/lib/db';

const environment = process.env.NODE_ENV;
const dbUrl = process.env.POSTGRES_URL ?? process.env.DATABASE_URL;

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  try {
    const db = getPool();
    let { rows } = await db.query('select now() as now');
    const now = rows[0]?.now;
    ({ rows } = await db.query('SELECT artist FROM albums LIMIT 1'));
    const artist = rows[0]?.artist;
    res.status(200).json({ time: now, artist: artist, message: `Sparks Database connection successful. Running in ${environment}.
DATABASE_URL: ${dbUrl}` });
  } catch (err) {
    res.status(500).json({ error: 'Database connection failed',
       details: (err as Error).message, message: `Sparks Database connection failed. Running in ${environment}. DATABASE_URL: ${dbUrl}` });
  }

}
```

Testing the Local App Against the Deployed *music* DB.

Toggle connection in *.env.development.local* to point to Vercel. Running the application locally, test *api/db-check*. You should see success

---

Take a screenshot showing Vercel music connection success from the local application. The connection string should be Vercel, and the artists query should show success.

Testing the Deployed Connection

Follow the procedure to push your updates to GitHub. *Ongoing Workflow*

Testing the Deployed App Against the Deployed *music* DB.

Run your Next.js application from the Vercel deployment page. Test *api/db-check*. You should see success.

Take a screenshot showing Vercel music connection success from the deployed application. The connection string should be Vercel, and the artists query should show success.

Next.js API Creation

If your deployed application can connect to the Vercel PostgreSQL *music* database, you have everything in place to develop a React.js API.

Unzip the provided *api.zip* under your projects *app* folder. The resulting structure should look like this:

```
app
├── api
│   ├── albums
│   │   ├── [slug]
│   │   │   └── route.ts
│   │   ├── search
│   │   │   ├── artist
│   │   │   │   └── [searchTerm]
│   │   │   │       └── route.ts
│   │   │   └── description
│   │   │       └── [searchTerm]
│   │   │           └── route.ts
│   │   └── route.ts
│   └── artists
│       └── route.ts
```

Testing the API Endpoints

Locally

1. Toggle connection in *.env.development.local* to localhost settings.
2. Run the updated Next.js application locally.
3. Duplicate your existing Postman collection of Music API Http requests.
    a. Rename the collection to indicate that it is a Next.js API local test set.
4. Next.js API routing defaults to an */api* prefix for API routes. The Express API did not follow this convention. Update all Postman tests in this collection to use the */api prefix*.
5. *Test all endpoints*

Take screenshots of successful: GET, POST, GET (find by Id) from Postman against the localhost URL.

Vercel Deployment

1. Run *vercel build* first.
    a. The Vercel cloud deployment build of Next.js runs extremely unforgiving lint checks (understand lint). This step will make sure your code will pass these tests.
        i. *vercel build* should become the first step before deploying code changes.
2. Push code changes to GitHub.
3. Run the deployed Next.js application.
4. Duplicate your existing Postman collection of Music API Http requests.
    a. Rename the collection to indicate that it is a Next.js API Vercel test set.
    b. The base url
5. Next.js API routing defaults to an */api* prefix for API routes. The Express API did not follow this convention. Update all Postman tests in this collection to use the */api prefix*.
6. *Test all endpoints*

Take screenshots of successful: GET, POST, GET (find by Id) from Postman

---

Understanding the Next.js API

The code provided implements a small REST- API using Next.js's *App Router*. Each endpoint lives under `app/api` and exports functions corresponding to HTTP methods; these functions run on the server and return `NextResponse` objects. Dynamic segments (folder names wrapped in brackets) allow parameters to be extracted from the URL.

Convention over configuration example: The Next.js *App Router* uses folder structure to define API routing. Therefore, no explicit route mapping.

Artists endpoint

File: `app/api/artists/route.ts`

- Exports a single `GET` function.
- Uses a database pool (`getPool`) to run `SELECT DISTINCT artist FROM albums ORDER BY artist`.
- Maps the result to an array of artist names and returns it with `NextResponse.json()`.
- Catches errors and returns a 500 status if something goes wrong.

This endpoint shows a basic read-only API route: it does not accept any parameters, simply queries the database, transforms the result, and responds with JSON.

Albums collection endpoint

File: `app/api/albums/route.ts`

- Exports `GET`, `POST` and `PUT` handlers.
- **GET:** Accepts an optional `albumId` query parameter. Without it, all albums are fetched; with it, only the album whose `id` matches is returned. It then queries the `tracks` table, groups tracks by album ID and attaches them to each album object. Invalid `albumId` values return a 400 response.
- **POST:** Parses a JSON body to create a new album. Within a transaction, it inserts the album record, then loops through any provided track objects and inserts them. On success it returns the new album's ID with status 201.
- **PUT:** Updates an existing album. Requires an `albumId` in the body; updates the album's fields and loops through provided tracks to update them. Uses a transaction to ensure atomic updates.

The pattern here highlights how to read query parameters via `request.nextUrl.searchParams` and how to perform multiple database operations within a transaction.

Albums by slug endpoint

File: `app/api/albums/[slug]/route.ts`

- Uses a dynamic segment (`[slug]`) so the value appears in the `params` object.
- **GET:** Treats the slug as an artist name; runs a case-insensitive query to fetch all albums by that artist, then attaches their tracks.
- **DELETE:** Treats the slug as a numeric album ID; parses it to an integer and runs `DELETE FROM albums WHERE id = $1`. Returns a 404 if no rows were affected.

Critique

Overloading the same parameter for both artist names and numeric IDs can be confusing; consider separate routes for `/api/artists/[artist]` and `/api/albums/id/[id]`.

Search endpoints

Files:

- `app/api/albums/search/artist/[searchTerm]/route.ts`

- `app/api/albums/search/description/[searchTerm]/route.ts`

Each exports a single `GET` handler. They extract `searchTerm` from `params`, perform a case-insensitive `ILIKE` query on the `artist` or `description` columns with `%searchTerm%`, fetch matching albums and their tracks, and return the list. Sanitising `searchTerm` and adding pagination (`?page=` and `?limit=`) would improve performance on large datasets.

Key Takeaways

- **Route handlers** live in the `app/api` directory; they replace the old pages-based API routes and support standard HTTP methods.
- **Dynamic segments** (`[slug]`, `[searchTerm]`) allow parameters to be passed via the URL and accessed in the handler's `params` argument.
- Use transactions for multi-step database operations and validate inputs before processing.
- Separate concerns—search endpoints, CRUD operations and deletion—into clear, predictable routes for maintainability.

Architectural Critique

Overall, database queries (`getPool().query()` calls) live inside the route handlers, mixing HTTP logic and data access.

A Repository Layer Would Help

- **Separation of concerns:** Next.js best-practice guidance for 2025 recommends keeping `/app/api` strictly for route handlers and placing external API or database logic in `/services`medium.com. The current code intermixes routing and SQL, leading to duplication when similar queries appear in multiple handlers.
- **Maintainability and security:** A DEV article on improving Next.js apps highlights moving data retrieval out of components to a server-only module and centralising queries for better maintainability and securitydev.todev.to. Centralised queries prevent exposing credentials and make updates easier.
- **Repository benefits:** The repository pattern abstracts and centralises data access operations, providing a consistent interface, improved testability and the flexibility to change data sourcesblog.alexrusin.com.

Suggested Approach

Create modules under `/lib` or `/services` (e.g., `albumRepository.ts`) that encapsulate database operations. Each method (`getAll`, `findByArtist`, `createAlbum`) runs SQL via `getPool` and is marked as server-only to prevent client accessdev.to. Route handlers then call these methods, focusing solely on HTTP concerns. This reduces duplication, eases testing, and aligns your project with modern Next.js architecture.

Project Status

We have Vercel cloud Music API.

Deliverables

Create a project report. The requirements for this report are in *Project Report Requirements:*

1. Document all create-next-app prompts and summarize their purpose. 4
2. After project creation modify a string in "page.tsx" to include your name. "npm run dev" in the project root folder. Follow the instructions to visit the page in the browser. 4
3. Take a screenshot of your GitHub repository with default Next.js app. Make sure your account name is shown along with the code. 5
4. Take a screenshot of the deployed default Next.js app. Make sure you show the address bar. Your name should be shown on the screen. This is already in place from the localhost step. 5
5. Take a screenshot of the deployed default Next.js app. The screenshot should show a difference from the earlier shot. Call out the difference. 6
6. Test the 'db-check URL. Take a screenshot of your browser. This step should show failure to connect to the local PostgreSQL database. 9
7. Test the 'db-check URL. Take a screenshot of your browser. This step should show connection success to the local installation of Vercel. Make sure to modify the results so that your name is shown. 11
8. Take a screenshot of your browser. Make sure the URL is part of the screenshot, it should show a Vercel address. This step should show failure to connect. 12
9. Take a screenshot of your application running locally and connecting successfully to the'neondb' on Vercel. 13
10. A Screenshot of /api/db-check reporting music database connection success on localhost. 13
11. Take a screenshot of the music database on Vercel in pgAdmin 14
12. Take a screenshot showing Vercel music connection success from the local application. The connection string should be Vercel, and the artists query should show success. 15

13. Take a screenshot showing Vercel music connection success from the deployed application. The connection string should be Vercel, and the artists query should show success. 16
14. Take screenshots of successful: GET, POST, GET (find by Id) from Postman against the localhost URL. 17
15. Take screenshots of successful: GET, POST, GET (find by Id) from Postman 17

Rubric

Points will be evenly distributed over the 15 Deliverable requirements.